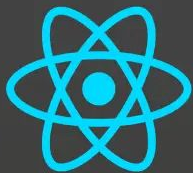


Selamat datang



WPU Ramadhan Camp

Kajian React.js Dari Dasar Hingga Mahir



sponsored by:



WPU Ramadhan Camp

Kajian React JS Dari Dasar Sampai Mahir

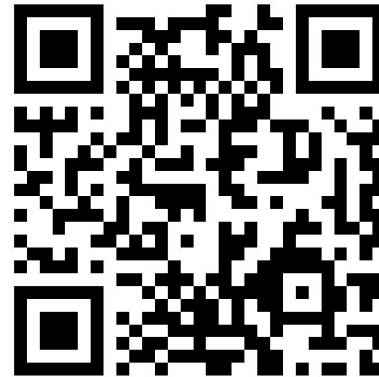
Rules Bootcamp

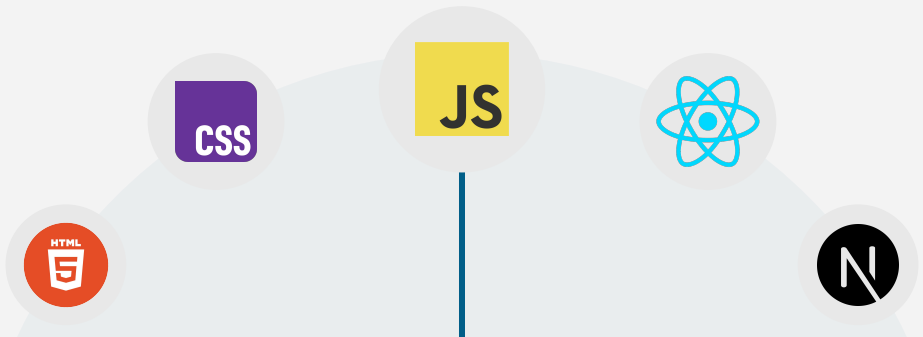
- Masuk ke zoom beberapa menit sebelum waktu yang dijadwalkan untuk menghindari ketertinggalan materi
- Gunakan Virtual Background WPU Ramadhan Camp
- Jika memungkinkan, aktifkan kamera untuk meningkatkan interaksi dan partisipasi
- Matikan mikrofon Anda (mode senyap) saat tidak berbicara untuk menghindari gangguan suara latar
- Apabila memiliki pertanyaan kita menyediakan slido supaya apabila teman-teman punya pertanyaan yang sama tinggal upvote pertanyaan yang lain yang sudah diajukan oleh rekan yang lain

pertanyaan

slido

Join at
slido.com
#4236 804





Asynchronous Javascript

Sandhika Galih

pre-requisite

JS

JAVASCRIPT LANJUTAN

1.1 Object Revisited

JAVASCRIPT LANJUTAN

Web Programming UNPAS • Course

Public

27 videos Last updated on Oct 8, 2022

Resume

Comments

No description

Sort

JAVASCRIPT LANJUTAN

1.1 Object Revisited

17:31

JAVASCRIPT LANJUTAN

1.2 Object.create()

10:39

JAVASCRIPT LANJUTAN

1.3 Prototype

17:33

JAVASCRIPT LANJUTAN

2.1 Execution Context, Hoisting & Scope

26:38

JAVASCRIPT LANJUTAN

2.2 Closures

20:43

JAVASCRIPT LANJUTAN | 1.1 OBJECT (Revisited)

Web Programming UNPAS • 355K views • 5 years ago

JAVASCRIPT LANJUTAN | 1.2 Object.create()

Web Programming UNPAS • 85K views • 5 years ago

JAVASCRIPT LANJUTAN | 1.3 Prototype

Web Programming UNPAS • 87K views • 5 years ago

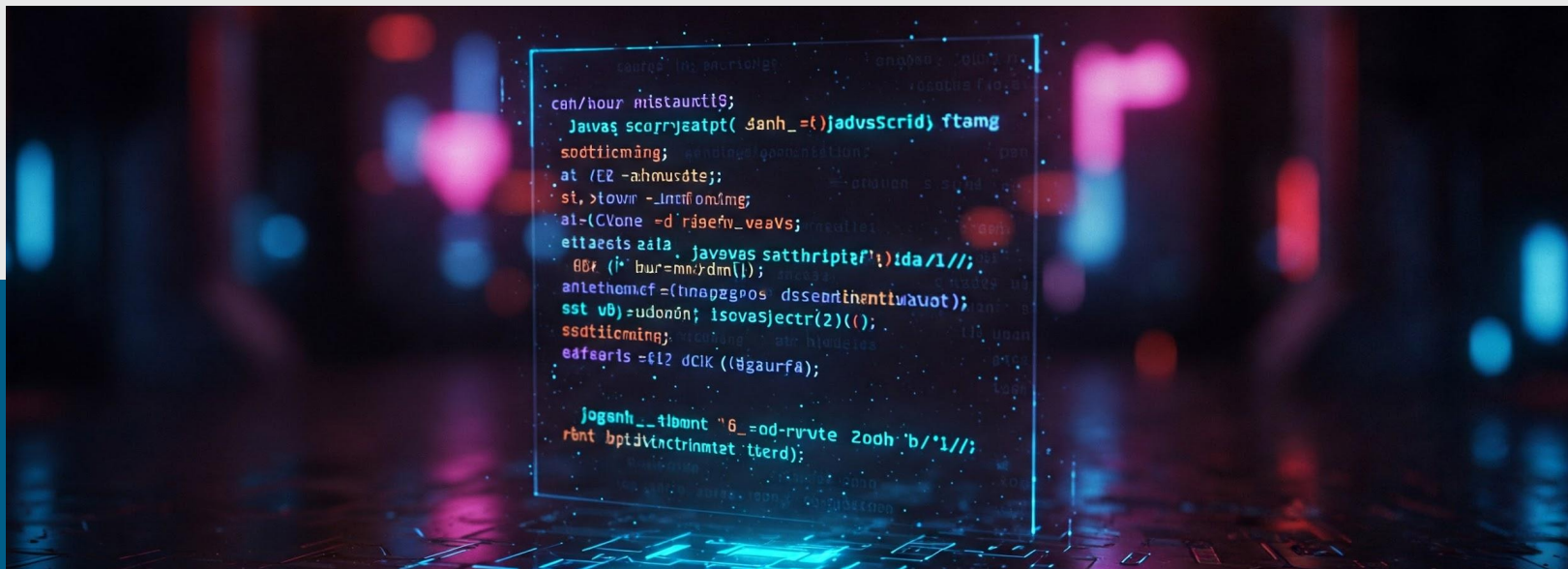
JAVASCRIPT LANJUTAN | 2.1 Execution Context, Hoisting & Scope

Web Programming UNPAS • 80K views • 5 years ago

JAVASCRIPT LANJUTAN | 2.2 Closure

Web Programming UNPAS • 72K views • 5 years ago

WPU Ramadan Camp 2025 | Kajian ReactJS Dari Dasar Hingga Mahir



01 First-class Function

First-class function ?

kemampuan untuk menggunakan **function**
sebagai **nilai / value**

```
1  ✓ let helo = function () {  
2    | console.log( 'Hello World' );  
3    };
```

First-class Function

- Dapat disimpan di dalam variabel
- Dapat diteruskan / dikirim sebagai argumen ke function lain (*callback*)
- Dapat dikembalikan dari fungsi lain
- Sebagai object yang memiliki property dan method

First-class Function

disimpan dalam variabel

```
1  ✓ let hello = function () {  
2    console.log( 'Hello World' );  
3  };
```

First-class Function

dikirim sebagai argumen

```
1  function sapa(nama, sapaan) {  
2    sapaan(nama);  
3  }  
4  
5  function salam(nama) {  
6    console.log('Halo, ' + nama + '!');  
7  }  
8  
9  sapa('Sandhika', salam);
```

First-class Function

dikembalikan dari fungsi lain

```
1  function buatPengali(pengali) {  
2    return function (angka) {  
3      return angka * pengali;  
4    };  
5  }  
6  
7  const pengaliDua = buatPengali(2);  
8  console.log(pengaliDua(5));
```

First-class Function

object yang memiliki property

```
1  function contoh() {  
2    contoh.properti = 'Properti Fungsi';  
3  }  
4  
5  contoh();  
6  console.log(contoh.properti);
```


Perbedaan Jenis Function

```
1 // function declaration
2 function validasiPassword(password) {
3   return password.length >= 8;
4 }
5
6 // function expression
7 const validasiPassword = function(password) {
8   return password.length >= 8;
9 };
10
11 // arrow function
12 const validasiPassword = (password) => password.length >= 8;
```



— 02 Callback Function

callback function ?

fungsi yang dikirim sebagai **argumen** ke
fungsi lain.

Callback Function

```
1 function sapa(nama, sapaan) {  
2   sapaan(nama);  
3 }
```

```
5 function salam(nama) {  
6   console.log('Halo, ' + nama + '!');  
7 }
```

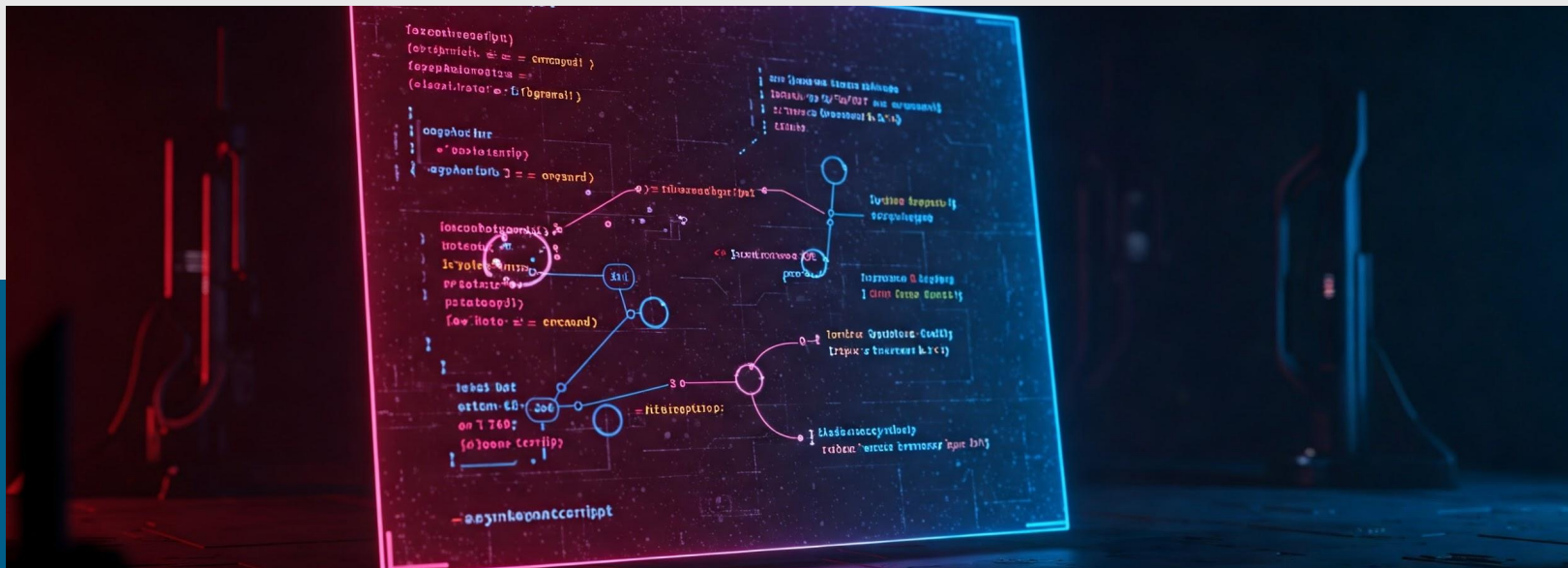
```
9 sapa('Sandhika', salam);
```

Callback
Function

Callback Function

- Fungsi yang dikirim sebagai argumen ke fungsi lain
- Fungsi callback kemudian dieksekusi setelah tugas tertentu selesai
- Memungkinkan penanganan operasi **asynchronous**
- Memungkinkan kode untuk berjalan **non-blocking**
- Yang berarti program dapat terus berjalan sambil menunggu tugas asinkron selesai

asynchronous ?



03

Asynchronous Javascript

**"javascript is a single-threaded,
non-blocking, asynchronous and
concurrent language."**

single-threaded

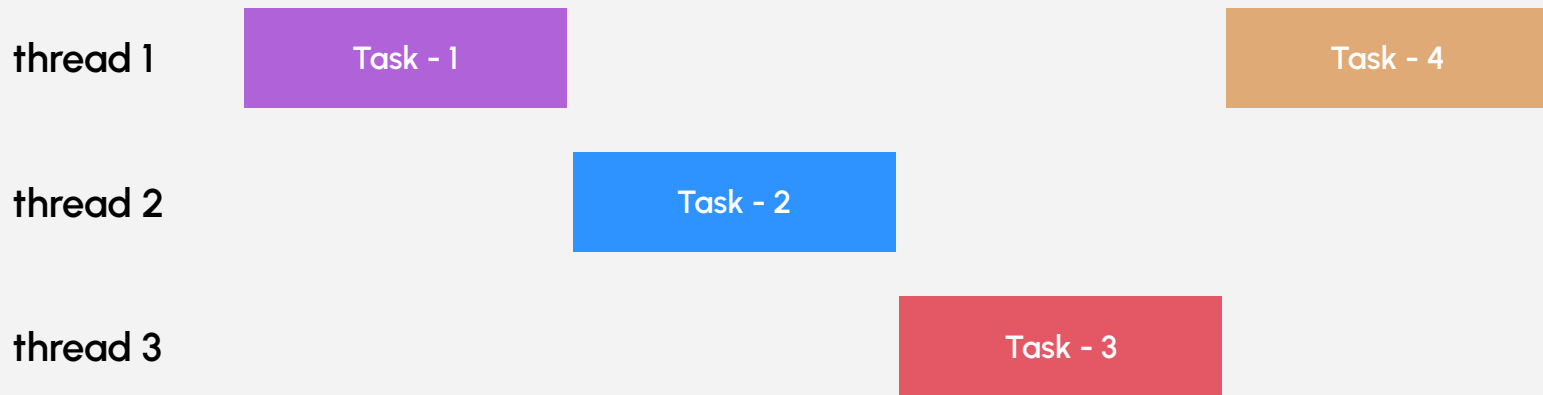
program hanya mampu menjalankan **satu perintah / *task*** dalam satu waktu.

Single-threaded

thread 1



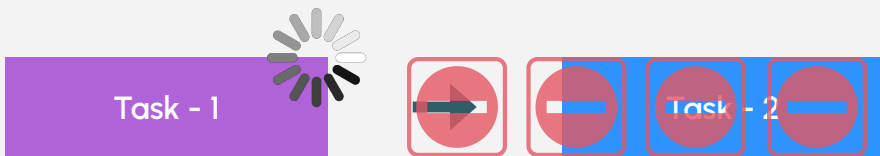
Multi-threaded



blocking

eksekusi code **terhenti** karena sebuah ***task***
berjalan **terlalu lama**

Blocking



Non-blocking



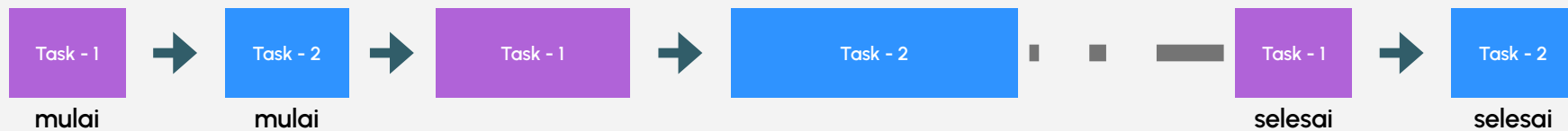
asynchronous

melakukan banyak tugas **secara bersamaan**
tanpa menghambat **thread utama**

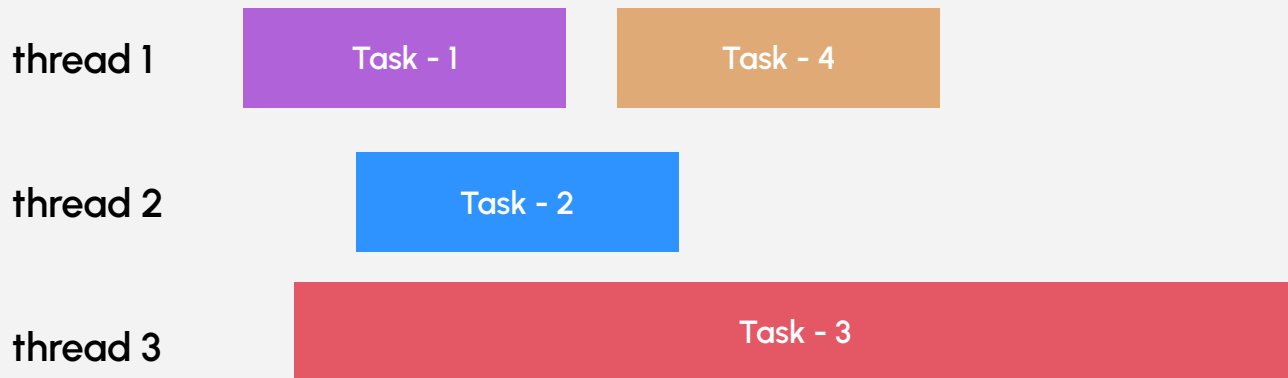
Synchronous



Asynchronous single-threaded



Asynchronous multi-threaded



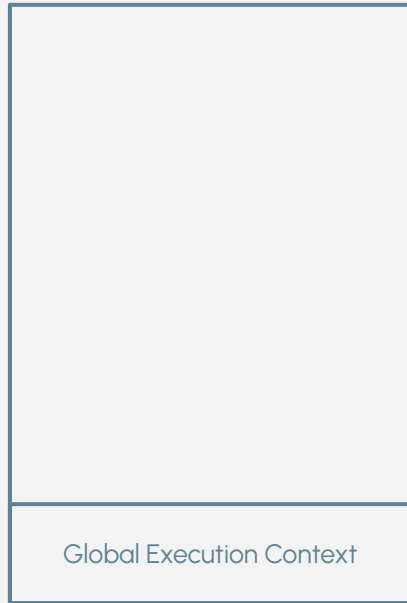
Asynchronous + Single-threaded = Concurrency

Asynchronous + Multi-threaded = Parallelism

mari kita pelajari **lebih dalam**

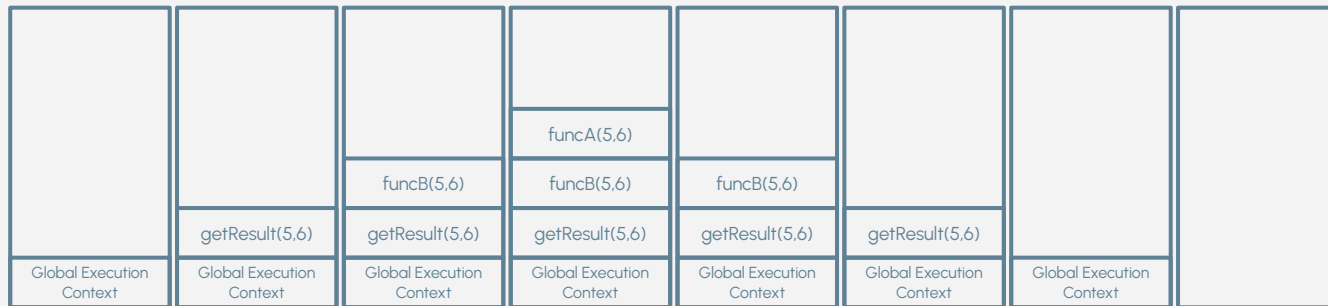
masih ingat **call stack** ?

Call Stack



Call Stack

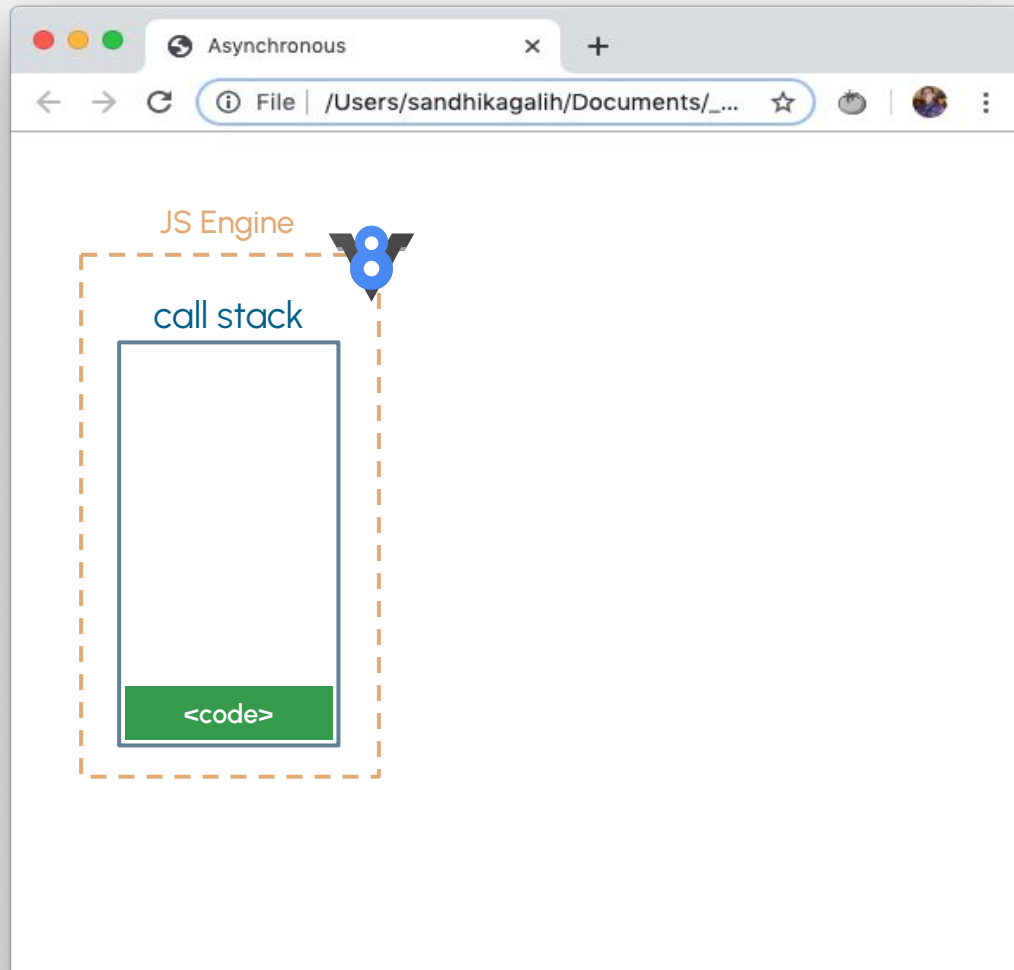
```
1 function funcA(m, n) {  
2   return m * n;  
3 }  
4  
5 function funcB(m, n) {  
6   return funcA(m, n);  
7 }  
8  
9 function getResult(num1, num2) {  
10  return funcB(num1, num2);  
11 }  
12  
13 var res = getResult(5, 6);  
14  
15 console.log(res);
```

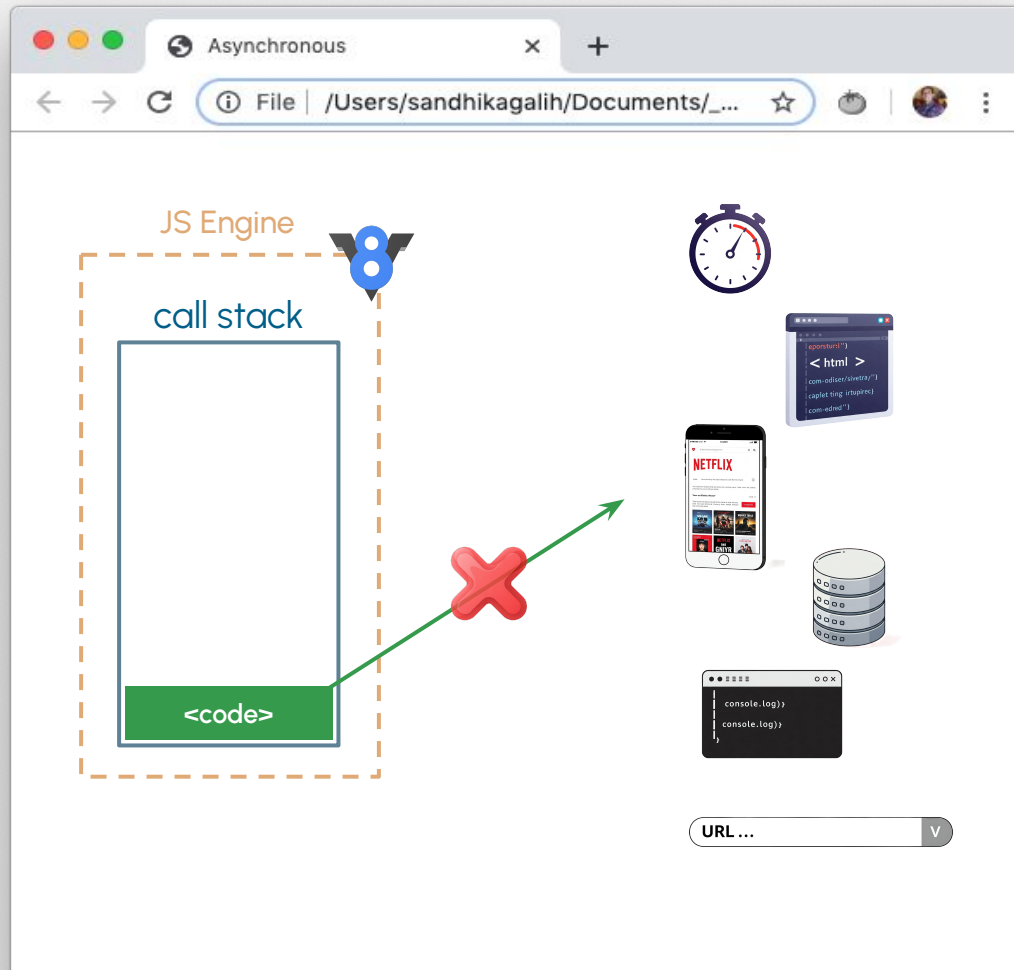


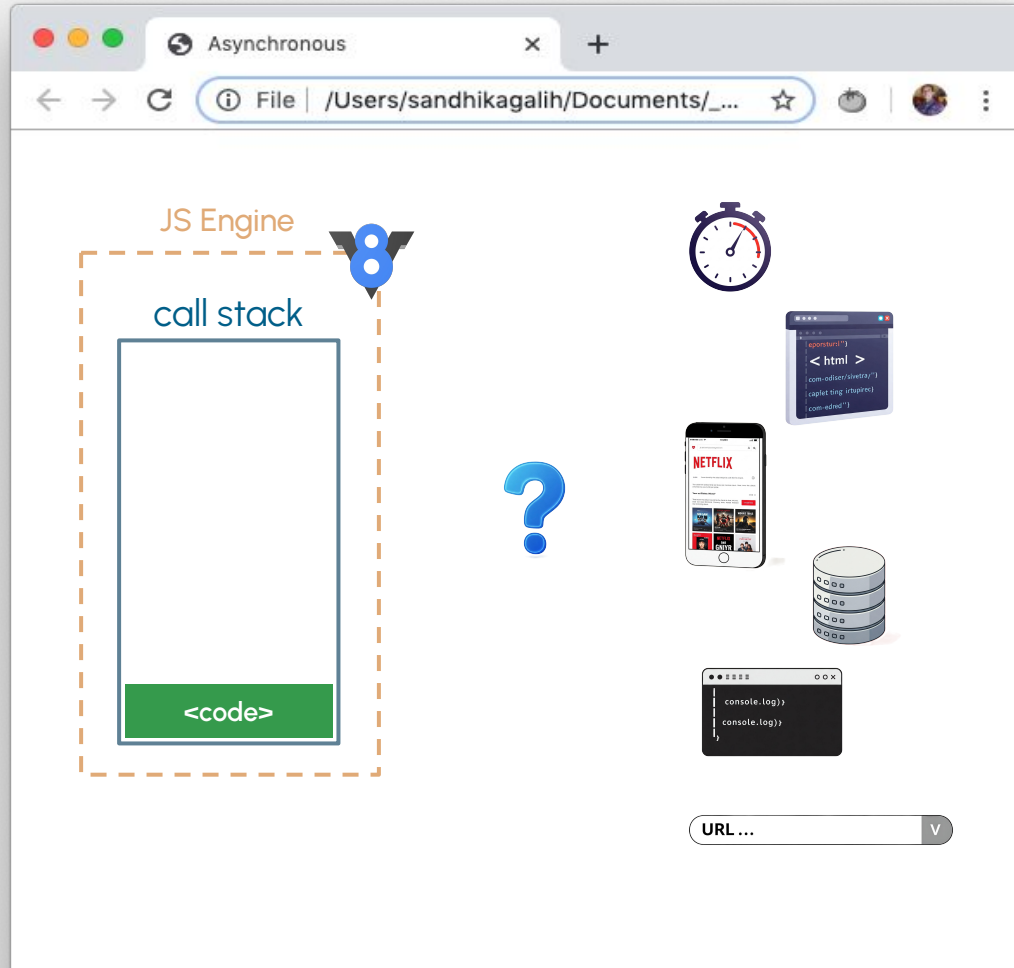
semua task yang masuk ke dalam **call stack**,
akan **langsung dieksekusi**

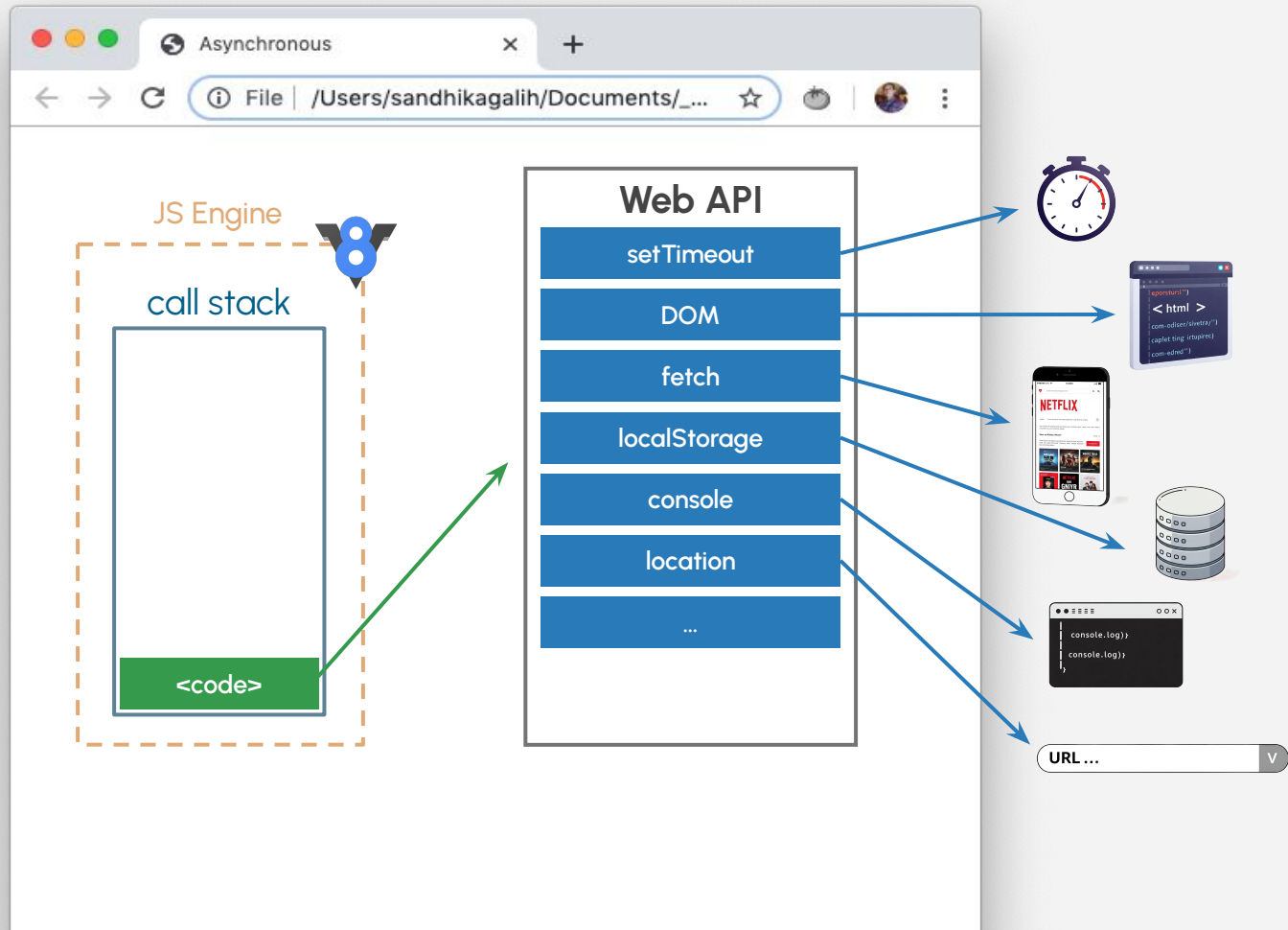
bagaimana jika kita menambahkan **timer** ?
agar **menunggu** dulu sebelum task dikerjakan

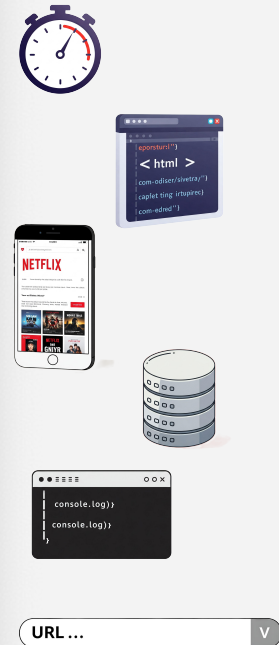
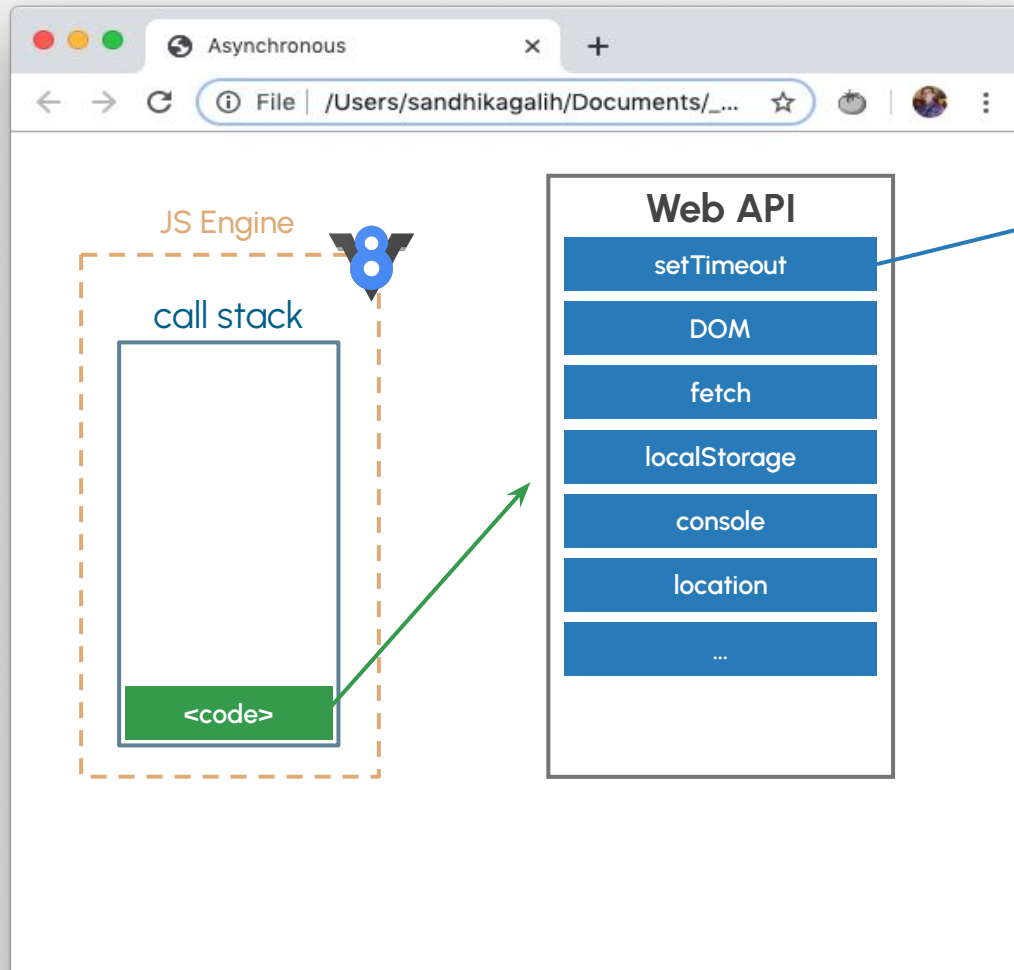
kita butuh **super power**
yang tidak dimiliki oleh **call stack**





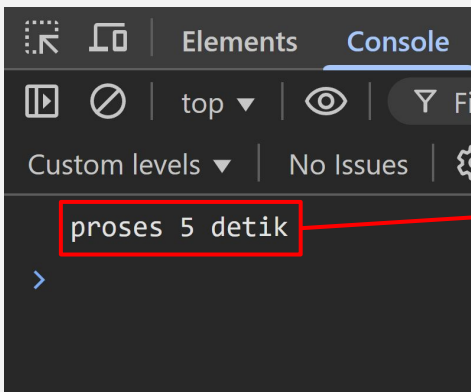






contoh

```
1  setTimeout(function () {  
2    console.log('proses 5 detik');  
3  }, 5000);
```



muncul setelah
5 detik

contoh lain

```
1 console.log('mulai');  
2 setTimeout(function(){  
3   console.log('proses 5 detik');  
4 }, 5000);  
5 console.log('selesai');
```

```
mulai  
selesai  
proses 5 detik  
>
```

muncul setelah
5 detik

??

coba kita ubah code nya

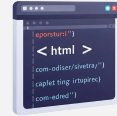
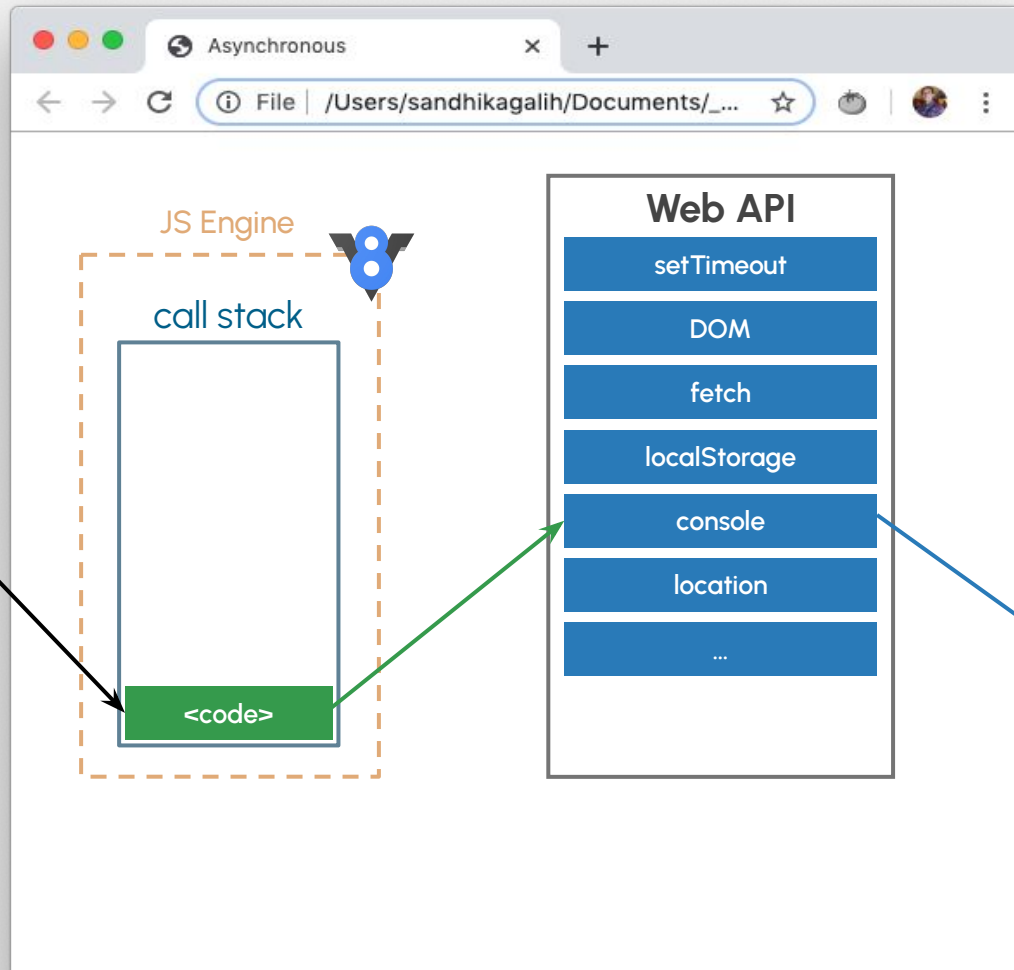
```
1 console.log('mulai');  
2 ✓ setTimeout(function () {  
3   console.log('proses 5 detik');  
4 }, 0);  
5 console.log('selesai');
```

```
mulai  
selesai  
proses 5 detik  
>
```

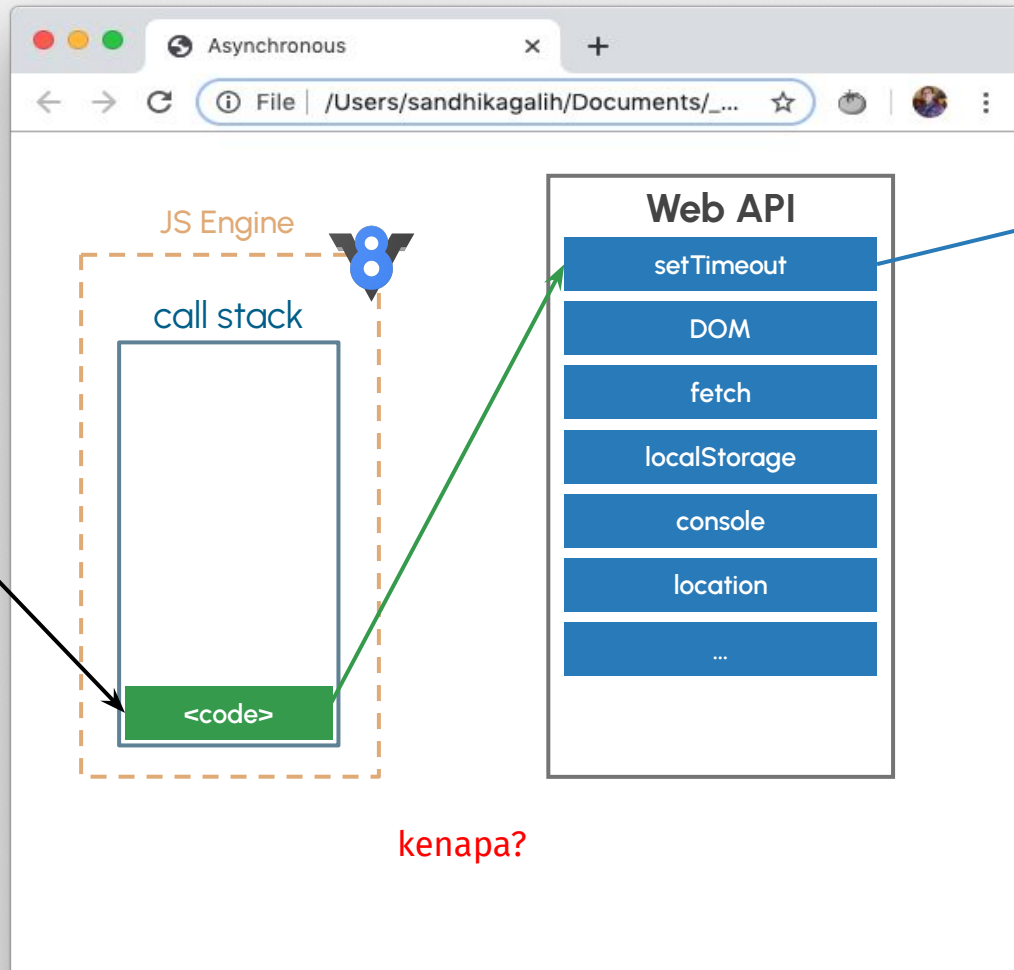
muncul setelah
0 detik

???.!!

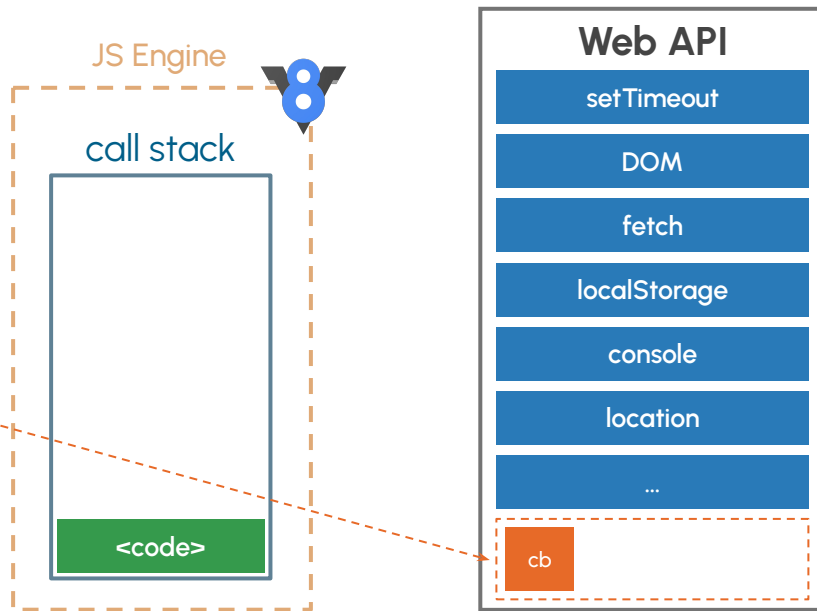
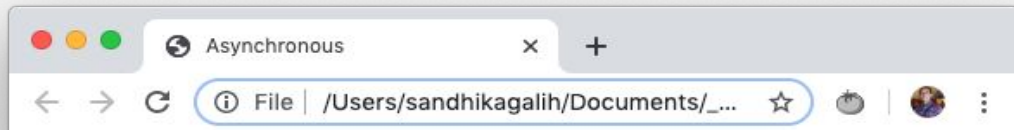
mari kita lihat **cara kerja** nya



URL ...

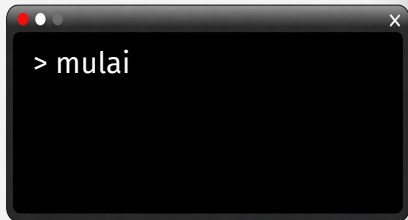


kenapa?

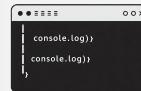
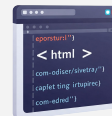


karena setTimeout()
merupakan asynchronous callback

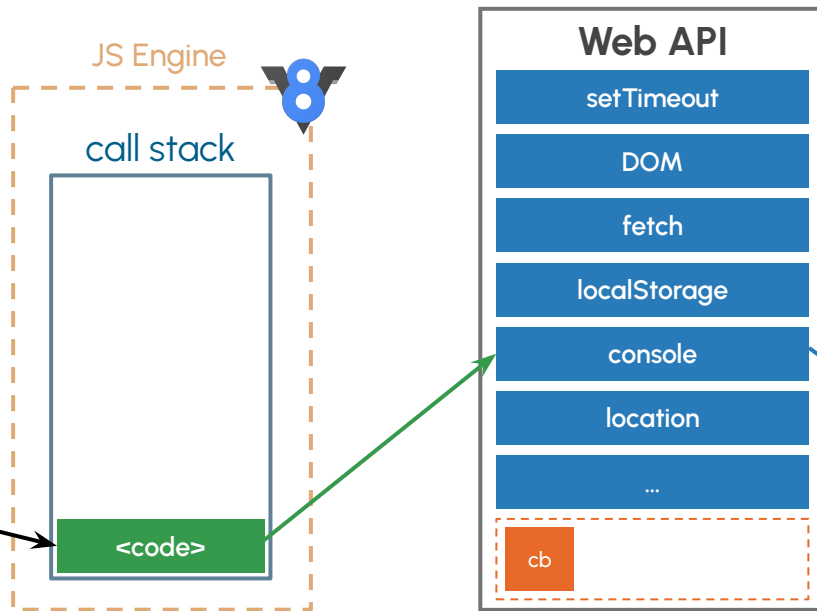
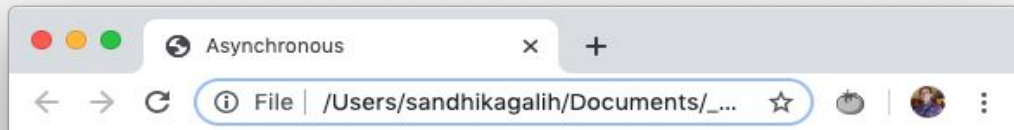
```
1 console.log('mulai');  
2 setTimeout(function ()-{  
3   console.log('proses 5 detik');  
4 }, 5000);  
5 console.log('selesai');
```



selanjutnya gimana?



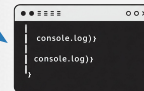
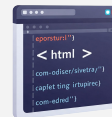
URL ...



terus callback gimana?

```
1 console.log('mulai');
2 setTimeout(function ()-{
3   console.log('proses 5 detik');
4 }, 5000);
5 console.log('selesai');
```

```
> mulai
> selesai
```



URL ...

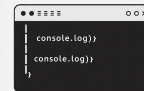
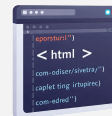
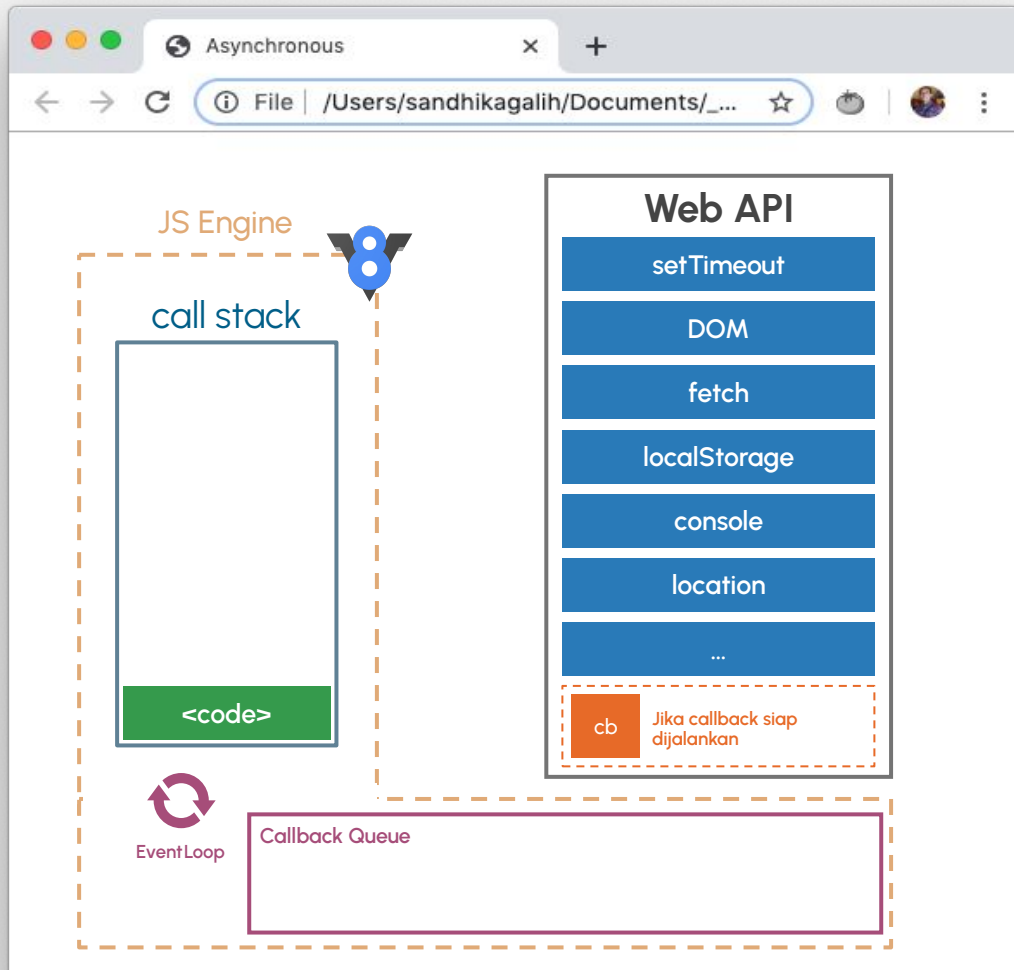
event loop & callback queue

Event Loop

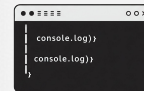
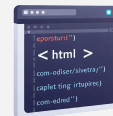
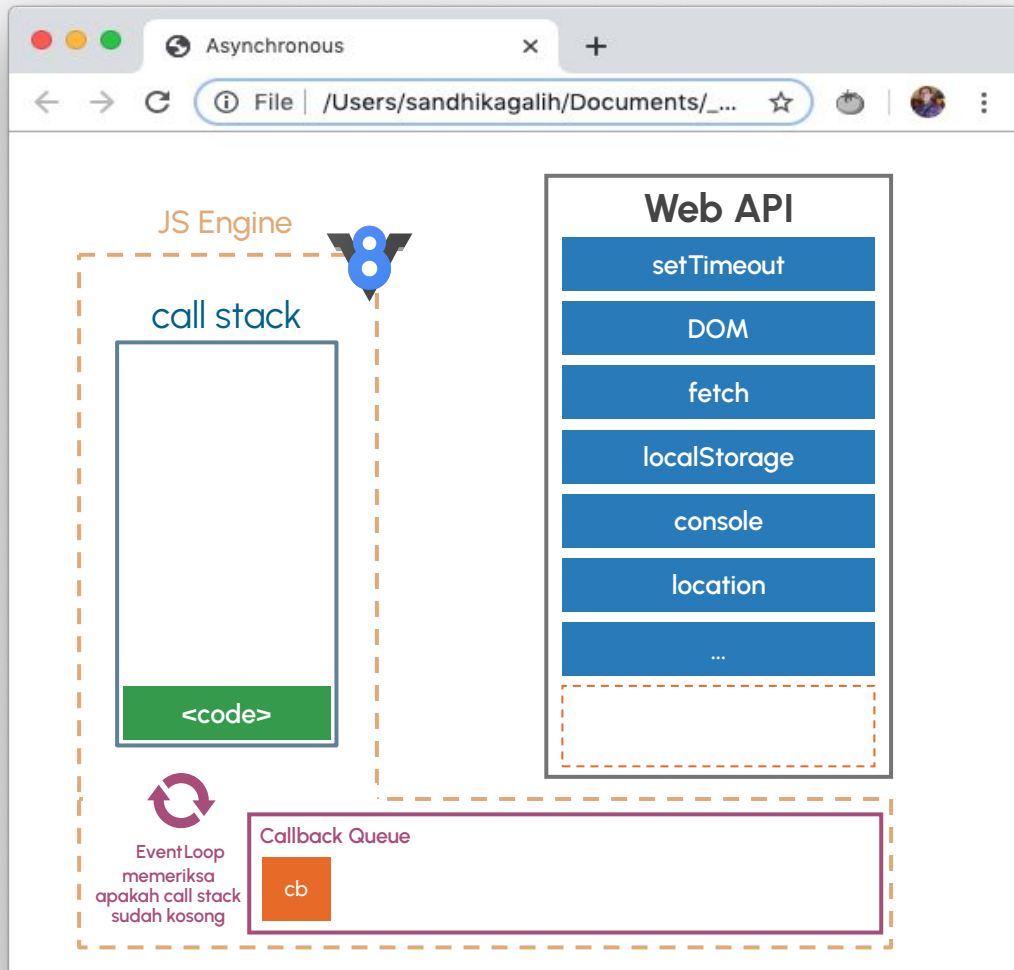
- Mekanisme yang digunakan oleh Javascript untuk menangani task **asinkron** (termasuk ***asynchronous callback***)
- Bertugas memantau **call stack** dan **antrian callback** (*callback queue*)
- Ketika tugas asinkron **selesai**, callback-nya ditambahkan ke **callback queue**
- Setelah itu, **event loop** akan memindahkannya ke **call stack** ketika call stack kosong

Callback Queue

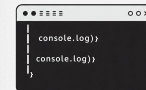
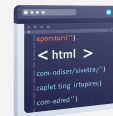
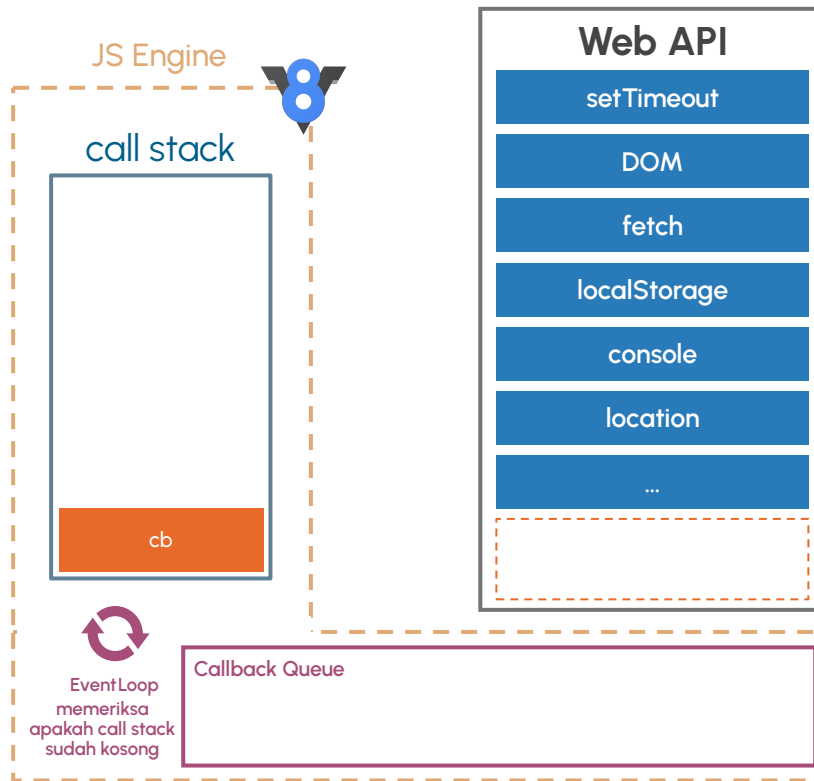
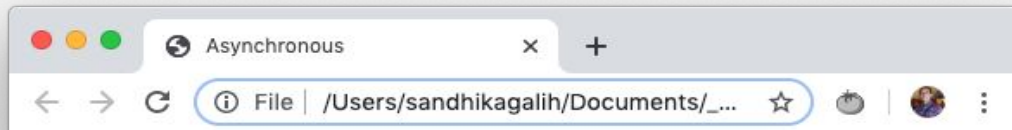
Antrian dimana **callback** disimpan sampai dapat dijalankan oleh **event loop**.



URL ...



URL ...



URL ...

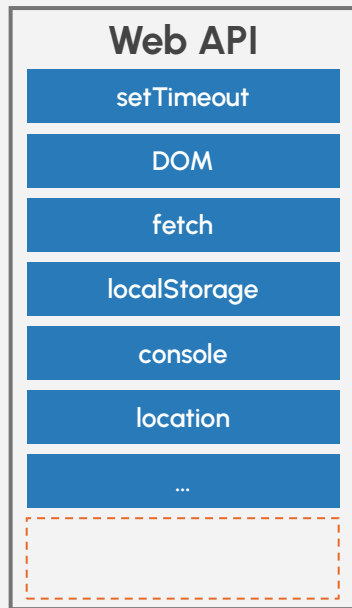
```
1 console.log('mulai');  
2 setTimeout(function(){  
3   console.log('proses 5 detik');  
4 }, 5000);  
5 console.log('selesai');
```

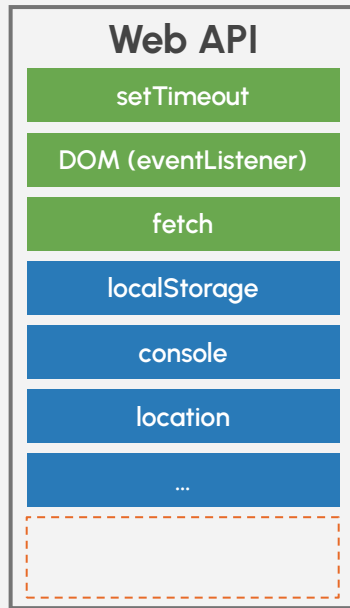
```
mulai  
selesai  
proses 5 detik  
>
```

muncul setelah
5 detik

mari kita cek secara **real time**

latentflip.com/loupe
atau
jsv9000.app





bersifat asynchronous

eventListener

```
1 console.log('mulai');
2 const tombol = document.querySelector('button');
3 ▼ tombol.addEventListener('click', function () {
4   | console.log('Tombol diklik!');
5   | });
6 console.log('selesai');
```

mulai

selesai

Tombol diklik!

muncul setelah
tombol diklik

fetch()

```
1 console.log('mulai');
2 fetch('https://wpu-cafe.vercel.app/api/menu')
3   .then((response) => {
4     return response.json();
5   })
6   .then((data) => {
7     const menu = data.data.map((menu) => menu.name);
8     console.dir(menu);
9   });
10 console.log('selesai');
```

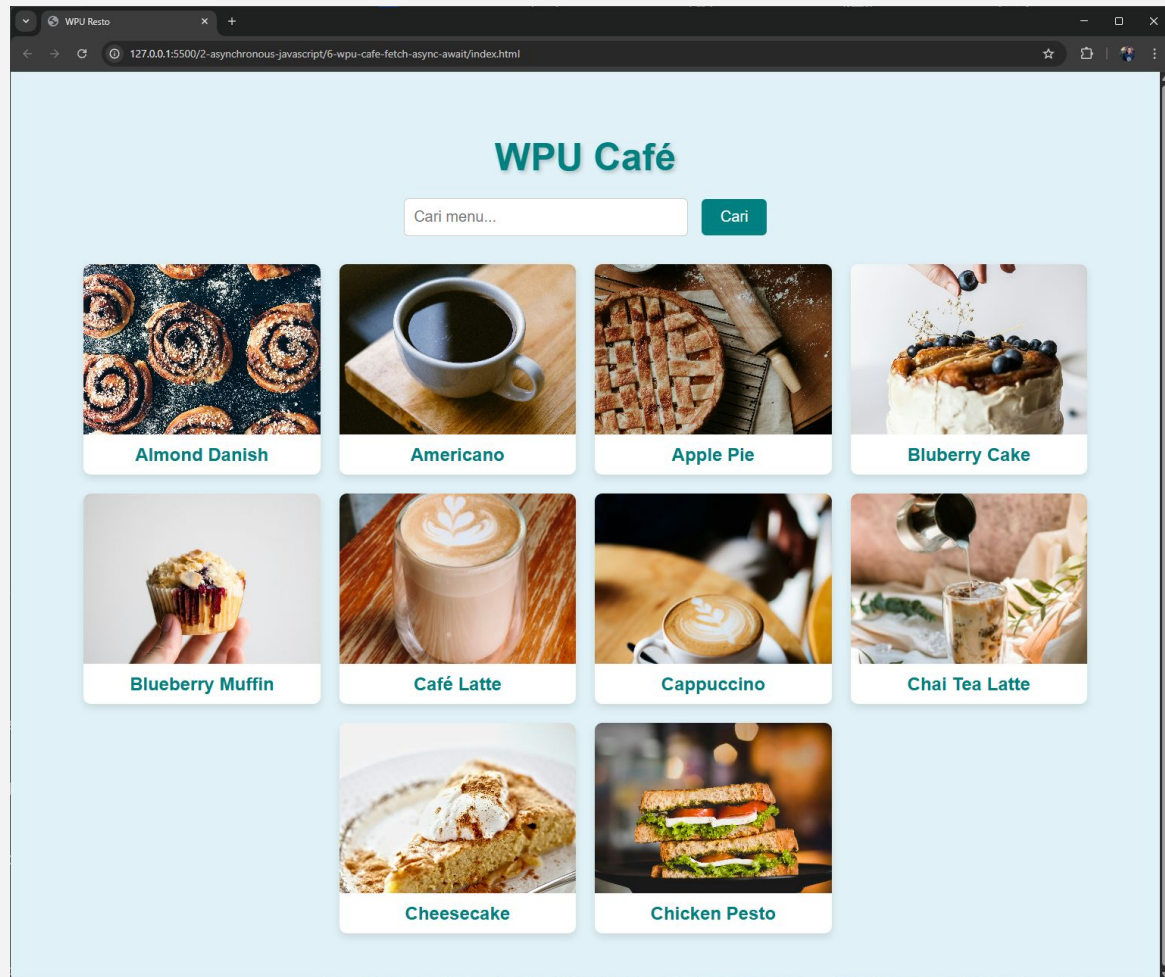
mulai

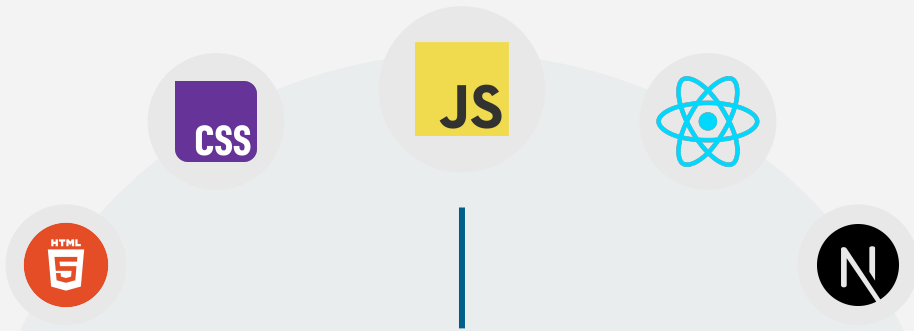
selesai

▼ Array(10) 

- 0: "Almond Danish"
- 1: "Americano"
- 2: "Apple Pie"
- 3: "Bluberry Cake"
- 4: "Blueberry Muffin"
- 5: "Café Latte"
- 6: "Cappuccino"
- 7: "Chai Tea Latte"
- 8: "Cheesecake"
- 9: "Chicken Pesto"

muncul setelah data
selesai didapatkan





Terima kasih!

wpucourse.id
@sandhikagalih